For example, suppose you want to store the Frequent Flyer member details between steps. You could do this as follows:

```
public class FrequentFlyerHelper {
    private FrequentFlyer frequentFlyer;
    public void setFrequentFlyer(FrequentFlyer frequentFlyer) {...}
    public FrequentFlyer getFrequentFlyer() {...}
}
```

You could then inject this class into your step definitions, as shown here:

```
public class TransferSteps {

    private final FrequentFlyerHelper frequentFlyerHelper;

    public TransferSteps(FrequentFlyerHelper frequentFlyerHelper) {
        this.frequentFlyerHelper = frequentFlyerHelper;
    }

    @Given("^(.*) is a Frequent Flyer member$")
    public void a_Frequent_Flyer_member(String name) {
        FrequentFlyer member = FrequentFlyer.called(name);
        frequentFlyerHelper.setFrequentFlyer(member);
    }
}
```

Cucumber will inject the helper class.

You can then use this class to share data across steps.

6.5.7 Pending steps and step outcomes

In Cucumber, the recommended way to say that a step implementation is still a work-in-progress is to throw a `PendingException`:

```
import cucumber.api.PendingException;
...
    @Given("^the (flying|travelling) distance between (.*) and (.*) is (\\d+)
     km$")
    public void define_flying_distance(String flightMode,
                                       String departure,
                                       String destination,
                                       int distance) {
        throw new PendingException("Not finished yet");
    }
```

This will cause this step and the scenario as a whole to be flagged as pending.

Cucumber-JVM can do a great deal more than what we've touched on here. But this should give you enough to get you started with Cucumber-JVM, or to understand a set of existing automated acceptance criteria implemented this way. You can learn more about Cucumber-JVM at http://cukes.info/.

6.6 Automating scenarios in Python with Behave

Python is a popular, general-purpose, open source dynamic language, and there are several Gherkin-based BDD tools available for the Python language. Cucumber itself can be used to run scenarios written in Python, though it uses a Python interpreter embedded inside a Ruby process, which can be brittle. For those who prefer